

BEAT CLAUDE · SINGLE GRAIN · CONFIDENTIAL



BLACK LABEL

Real-Time Analytics Pipeline

Engineer 004 — System Design Submission

Michael Barber · mthburnsbarber@gmail.com

Brief version: 2026-07 · Submitted 2026-08-03

Fixture sha256: 1aeb24b415009e89fcf8acb5a178410faf216dc17b16920d9849ecc8bbb24235

Artifact: beat-claude-engineer-004/ — detector, 20 tests, benchmark, cost model

WRITTEN ANSWER

0. The number that reframes the brief

[Observed, brief] 50M events/day is [Estimated, arithmetic] **579 events/sec** average. At the brief's [Observed] 10x spike that is **5,787/s**. [Observed, fixture] mean event size is 230 bytes, so the firehose is [Estimated] **11.5 GB/day raw, ~1.3 GB/day compressed**.

That is not a scale problem. I benchmarked the full normalize-and-classify step in CPython — the slowest plausible implementation — at [Benchmarked] **112,280 events/sec/core** (median of 3 runs, 100k events, `out/bench_ingest.json`), p99 7.4→21.8 µs/event. The entire 10x spike is [Estimated] **0.05 of one core** of transform work.

So the current system is not falling over because 50M/day is heavy. It is falling over because it has no ingest contract: no idempotency, no quarantine, no backpressure, no tenant validation. The fixture proves this — every seeded defect is a contract defect, not a capacity defect. I am therefore spending the design on **correctness under load and migration safety**, and deliberately *not* on exotic scale-out.

1. Architecture and technology choices

See `ARCHITECTURE.md` for the four diagrams (data flow, verdict machine, migration, deletion).

Edge (unchanged for customers). The SDK cannot change, so the existing hostname, path and response contract stay exactly as they are. A new stateless collector on **ECS Fargate** sits behind the **existing ALB listener**. It authenticates, resolves `tenant_id`, applies a per-tenant quota, and returns 200 **only after** the durable enqueue is acknowledged. If the enqueue fails it spills to local disk and SQS; if that fails it returns 429 and the SDK's existing retry does the rest. *Rejected:* API Gateway + Lambda (per-request cost and cold-start jitter at [Estimated] 579/s sustained, and it hides the connection-level backpressure I need); writing straight to the database (no replay).

Durable log: Kinesis Data Streams, provisioned, [Estimated] 7 shards, [Assumed] 7-day retention. *Rejected:* Kinesis **on-demand** — it doubles capacity roughly every 15 minutes, which does not survive a step function from 1x to 10x at midnight on Black Friday; provisioned + scheduled pre-scale is the honest answer to a *scheduled* spike. *Rejected:* MSK/Kafka — better tool, but it costs a meaningful share of two engineers' attention to operate, and there is no Kafka expertise stated on this team. *Rejected:* SQS — no replay, and replay is the whole reliability story. Partition key is `tenant_id:anonymous_id`, which keeps one visitor's events ordered on one shard (needed for sessionization) and spreads tenants; the known hazard is a hot shard from one large tenant, handled by a per-tenant quota at the collector plus shard-level split alarms.

Ingest-edge contract — `pipeline/normalize.py`, the piece that exists as code because it is the riskiest. Its rule: *nothing is ever dropped*. Every line leaves as exactly one verdict — `accept`, `accept_flagged`, `duplicate`, `quarantine` — and a quarantine row retains the raw bytes and stays addressable by tenant and subject. [Observed, brief] "3% event loss" becomes "3% of events are in the quarantine table, here they are, here is who owns each class."

Serving is split by access pattern, on purpose. **ClickHouse** (`ReplacingMergeTree(event_id)`) for dashboards and behavioural segments — column scans over hundreds of millions of rows at an [Estimated] 5-second freshness. **DynamoDB** for the visitor profile the personalization decision reads at page render: that is an [Estimated] single-digit-millisecond point lookup in a user's critical path, and it must not share a fate with an analytical query. **ElastiCache Redis** for the [Assumed] 24-hour `event_id` dedupe set and hot segment counters. *Rejected:* Druid/Pinot (more moving parts than 2 engineers should own); Redshift/Athena (minutes, not seconds); Timesstream (weak on high-cardinality string dimensions); one store for both jobs (a dashboard scan would then be able to slow down a customer's homepage).

Lake: S3 + Apache Iceberg, partitioned `tenant_id/hour`. Iceberg specifically for **row-level deletes** — a GDPR erasure must not require rewriting a customer's history. Exports to Snowflake/BigQuery read the Iceberg tables per tenant.

Event model and identity. One canonical envelope (`schema_version`, `event_id`, `tenant_id`, `anonymous_id`, `user_id`, `type`, `ts`, `received_at`, `properties`), with legacy spellings translated at the edge forever, because the SDK cannot be fixed. Identity is a durable `(tenant_id, anonymous_id) → user_id` table, not stream state: `identify` events backfill prior anonymous events, and — proven by the fixture — a `user_id` can arrive with **no** `identify` event in the window, so the stitch table can never be rebuilt from one window of the stream.

2. Scale, reliability, migration

Guarantee, stated precisely. At-least-once from SDK to collector (the SDK retries; the fixture proves it). Exactly-once *effect* downstream, bought with idempotency keyed on `(tenant_id, event_id)` — never on `(visitor, type, ts)`, which would erase two genuine rapid clicks. Dedupe window is 24h in Redis; a duplicate arriving later is collapsed at read time by ClickHouse's `ReplacingMergeTree`. That seam is deliberate and documented, not hidden.

Sizing. [Estimated] 7 Kinesis shards covers 5,787/s with headroom; 6 Fargate tasks sized for the spike *held continuously*, not autoscaled into it, because autoscaling arrives after the spike does.

Degradation ladder — what gives first, in this order: (1) drop enrichment (geo/UA parsing); (2) widen the micro-batch [Assumed] 1s → 10s, so dashboards go from [Estimated] ~5s to ~15s; (3) pause warehouse export jobs; (4) shed bot-classified traffic (the `evt-0012`–`evt-0015` class); (5) 429 and let the SDK retry. **Never** shed the durable S3 path. Freshness is the sacrificial resource; durability is not.

Cost. `cost_model.py` (line-item, every input labelled) gives [Estimated] **\$4,952/month**, including a 30% contingency — **10% of the [Observed] \$50K ceiling**; a 2-day 10x spike month is [Estimated] \$5,258. The finding that matters: **budget is not the binding constraint, the two engineers are**. I would spend part of the ~\$45K headroom on ClickHouse Cloud instead of self-managed ClickHouse ([Estimated] \$+1–2K/month) to buy back the ops hours, and re-evaluate at month 6 with real numbers.

Migration — the constraint is that nothing may break for [Observed] 500+ tenants.

- *Weeks 1–2 — tee.* The collector dual-writes: existing pipeline untouched and still authoritative, plus the new Kinesis stream. Zero customer-visible change, zero SDK change.

- *Weeks 3–8 — parallel run.* New pipeline writes shadow tables. An hourly reconciler compares old vs new on event count by (tenant, hour), unique visitors, conversion events, and a 10k sampled `event_id` set. Ground truth is a separate **accept ledger** written by the collector at the moment of the 200.
- *Weeks 9–12 — per-tenant read cutover* behind a flag, starting with internal + 5 friendly low-volume tenants. Old pipeline keeps writing the whole time.
- *Months 4–6 —* decisioning, then warehouse exports, then decommission.

"Data accuracy verified" is **testable, not rhetorical**: per tenant, $|\text{count delta}| \leq 0.1\%$ for 14 consecutive days, **and** 100% of a 10k sampled `event_id` set present in both stores, **and** quarantine rate $\leq 0.1\%$ with every quarantine class assigned an owner. [Assumed] thresholds — I would tighten or loosen them after two weeks of real parallel-run data, not before.

Rollback. Pre-committed automatic triggers, all [Assumed] until parallel-run data exists: new-vs-old divergence $>0.5\%$ over 15 min for any tenant, or p99 end-to-end $>10\text{s}$ for 10 min, or quarantine rate $>0.5\%$. The flag reverts and on-call is paged. Rollback is a read-path flag flip, not a data movement — which is the *only* reason it can be automatic — and it stays available until the old pipeline is decommissioned.

What I can and cannot prove about loss. The accept ledger proves zero loss from the collector's 200 onward. Browser → collector cannot be proven without SDK sequence numbers, and the SDK cannot change. So the honest claim is: **loss after the edge is measurable and provably zero; loss before the edge is bounded but unmeasured.** I would instrument it in the SDK's *next natural release* rather than call it solved.

3. What the fixture actually contains

`detect_anomalies.py` classifies all [Observed] 25 lines; `tests/test_normalize.py` asserts each class ([Observed] 20 tests, all passing, `out/test_run.txt`). Result: [Observed] 11 accept, 11 accept_flagged, 1 duplicate, 2 quarantine.

CLASS	EVENT IDS	HOW THE PIPELINE HANDLES IT
Malformed JSON	<code>evt-0020</code>	Quarantine with raw bytes + regex-scraped ids; never a dropped batch.
Duplicate <code>event_id</code>	<code>evt-0002</code> (lines 2, 5; +7.56s)	Idempotency on <code>(tenant, event_id)</code> ; counted, not stored.
Clock skew / negative transit	<code>evt-0005</code> (-47.2s), <code>evt-0006</code> (-3,900s)	Flagged, kept; <code>ts</code> used for user timeline, <code>received_at</code> for SLO.
Implausible future ts	<code>evt-0016</code> (+365 days)	Flagged, kept, excluded from time-series bucketing pending review.
Schema drift	<code>evt-0009</code> (<code>pageview</code> , <code>timestamp</code> , <code>page_path</code> , <code>ref</code> , no <code>received_at</code>)	Alias-translated at the edge, permanently. Also: no <code>received_at</code> ⇒ excluded from latency SLO.
Null tenant	<code>evt-0011</code>	Quarantine: unroutable, unbillable, undeletable per-tenant — a compliance hole, not a stats footnote.
PII in properties	<code>evt-0007</code> (<code>contact_email</code> , <code>phone</code>)	Detected pre-storage, encrypted with the per-subject key. Note it arrived anonymous , before <code>anon-3d0</code> → <code>u-7304</code> (<code>evt-0022</code>): governance cannot wait for identity.
Non-human burst	<code>evt-0012</code> – <code>evt-0015</code> (4 <code>page_views/50ms</code> , <code>scanner.example-bot.net</code>)	Flagged, stored, excluded from dashboards by default filter — reversible.
Client-computed aggregate	<code>evt-0019</code> (<code>count_today: 3</code>)	Stored as a property, never allowed to drive a segment.
Privacy request	<code>evt-0017</code> (GDPR, <code>u-1077</code>)	Full cascade — see below.
Retroactive identity	<code>anon-9f2</code> → <code>u-5511</code> (backfill <code>evt-0001</code> , <code>evt-0002</code>); <code>anon-c81</code> → <code>u-2209</code> (<code>evt-0004</code> , <code>evt-0005</code>); <code>anon-3d0</code> → <code>u-7304</code> (<code>evt-0007</code>)	Durable stitch table + backfill job.
Identity with no <code>identify</code>	<code>anon-52d</code> / <code>u-8842</code> , <code>anon-77a</code> / <code>u-1077</code>	Proves stitch state must outlive any stream window.

Two findings I would raise on day one:

The lost event is the valuable one. `evt-0020`, the only unparseable line, is a click on `#signup` — the highest-intent event in the sample. Loss is not random sampling; it correlates with newer, richer, higher-value payloads. "3% loss" and "3% of conversions lost" are not the same number, and the current system cannot tell you which one it has. [Observed, fixture: 1 of 25 lines.]

The GDPR request under-deletes. `evt-0017` names `u-1077`, but that subject's other event, [Observed, fixture] `evt-0006`, carries `user_id: null` and is reachable **only** via `anon-77a`. A deletion job keyed on `user_id` leaves it. Worse, `evt-0006` is also the -3,900s clock-skew event — so a pipeline that quarantines skewed events must sweep the quarantine table on erasure too. Both paths are asserted in `TestDeletionCascade`.

Three things I will not act on at face value:

1. `evt-0019`'s `count_today: 3`. It is the exact shape of the brief's "viewed pricing 3x" segment, and the server can observe [Observed, fixture] 1 `/pricing` view for `anon-9f2` (`evt-0001`). A client-computed counter is unverifiable, trivially spoofable, and would fire personalization on a number no one can reproduce. Segments are computed server-side from raw events, full stop.

2. **The bot burst.** [Observed, fixture] `evt-0012`–`evt-0015` are 4 of tenant `t-042`'s 11 events (36.4%) and 4 of its 6 `page_views`. I will not report a traffic number without a stated bot policy — but I also will not *delete* them, because the classifier will be wrong sometimes and changing a customer's historical numbers retroactively is worse than a filter they can toggle.
3. **The rates themselves.** $n = 25$ synthetic lines. I have deliberately published **no** percentages extrapolated from this fixture — the 36.4% above is scoped to the sample and labelled as such. Anyone converting 1/25 malformed into "4% malformed at 50M/day" is inventing a number. *(Also on the not-at-face-value list: `evt-0016`'s +1-year `ts` should be quarantined, not dropped — a blanket "drop future timestamps" rule silently deletes real events from skewed client clocks, of which `evt-0005` and `evt-0006` are two.)*

4. Trade-offs, risks, and scope

Optimizing for: provable non-loss, operability by two engineers, and a rollback that works. **Sacrificing:** sub-second latency (target is $p95 < 5s$, not $< 1s$); ad-hoc query flexibility at MVP (fixed dashboard query shapes only); physical tenant isolation — tenants share ClickHouse with logical isolation by partition key plus collector-level quotas, which is a **stated noisy-neighbour risk** accepted to stay inside two engineers' operating capacity.

MVP ([Observed, brief] month 3) excludes: cross-device identity graph, ML-derived segments, customer-facing SQL, sub-second decisioning, per-tenant retention policies, streaming (rather than 15-minute batch) warehouse export.

With more time/budget: ClickHouse Cloud from day one; Flink for real sessionization instead of Redis counters; per-tenant physical isolation for the [Assumed] top 10 accounts by volume; SDK sequence numbers to close the pre-edge loss gap; and a continuous synthetic-event canary per tenant measuring true end-to-end latency rather than inferring it.

OPERATING ARTIFACT

beat-claude-engineer-004/ — runs on stock Python 3.9+, stdlib only, no network, no credentials.	
FILE	WHAT IT IS
pipeline/normalize.py	The ingest-edge contract. Verdict machine, alias translation, PII detection, idempotency, bot/clock/aggregate classification.
detect_anomalies.py	Runs it over the fixture; per-line verdicts + cross-event findings (stitching, deletion cascade, aggregate contradiction). Exits 1 on any quarantine.
tests/test_normalize.py	20 tests, all passing. One per seeded anomaly class, plus the non-loss and idempotency invariants the design's claims rest on.
bench_ingest.py	Single-core throughput + latency benchmark.
cost_model.py	Line-item AWS cost model; every input labelled; change one number, everything re-derives.
ARCHITECTURE.md	Four Mermaid diagrams.
out/	Committed output of every run above.

EVIDENCE LOG

CLAIM	EVIDENCE	TIER
All 10 seeded anomaly classes detected, ids cited	out/anomaly_report.txt, out/anomaly_report.json; reproduce: \$ python3 detect_anomalies.py	3 — source records
Design invariants (no silent loss, idempotency, deletion cascade) hold	out/test_run.txt, 20/20 pass; reproduce: \$ python3 -m unittest discover -s tests -v	3 — source records
112,280 events/sec/core; 0.05 cores at 10x spike	out/bench_ingest.json measured on this machine; reproduce: \$ python3 bench_ingest.py	3 — measured run, method stated
\$4,952/month vs \$50K ceiling	out/cost_model.json, AWS us-east-1 public list prices 2026-08-03; reproduce: \$ python3 cost_model.py	2 — inspectable model, not a bill
Fixture identity: sha256 1aeb24b4...b24235	\$ shasum -a 256 fixtures/event_sample.jsonl; pinned in TestFixtureIntegrity	3 — checksum
Architecture behaves as drawn under 50M/day production load	None. Nothing here has run in production.	0 — claim only, stated as such

No Tier 4/5 evidence is offered: I have not run this system, and a before/after from a comparable production migration is not something I can hand you as a file today. I would rather label that honestly than dress an estimate up as a measurement.

NUMBER SOURCE LABELS

Every number above carries [Observed] (measured from the fixture or stated in the brief), [Benchmarked] (measured on this machine, or named external price list), [Estimated] (derived by stated arithmetic), or [Assumed] (placeholder to make the plan concrete). The load-bearing assumptions are: SDK batches ~10 events/POST (unverified — measure at the existing collector before build starts); 10% of events change durable visitor state; 60% trigger a decision read; 9:1 compression; 30% cost contingency. All five are single constants in cost_model.py.

AI USAGE DISCLOSURE

Tools: Claude (Opus 5) via Claude Code, in my own terminal, with the repo checked out locally.

What AI did: drafted the first pass of the detector and the test scaffold from my spec, wrote the Mermaid, and tightened prose. It also ran every command whose output appears here — nothing in out/ is transcribed by hand.

What I decided: the reframe (579/s is not a scale problem — spend the design on the contract); Kinesis provisioned over on-demand because of the 15-minute doubling ramp; the split serving store; Iceberg for row-level deletes; the degradation ladder and its ordering; the parallel-run gate thresholds; the refusal to extrapolate rates from n=25.

What I changed and checked: the first PII detector matched the bare key name and produced two false positives (evt-0016, evt-0019 — both custom event names, not people). I caught it in the output, narrowed the hint list, and left the mistake documented in the code plus a regression test, because it is the exact failure mode of key-name PII scanning at scale. I re-ran every number in this document from the committed scripts. I hand-read all 25 fixture lines before trusting any detector output — the evt-0006/evt-0017 deletion link and the evt-0020 -is-a-signup-click observation came from reading, not from the tool.

Known weak spots in the AI output: it initially wanted Flink and a Kafka cluster (correct for a 50-engineer team, wrong for two people); it wanted to *drop* future-dated events rather than quarantine them; and its first cost pass used DynamoDB on-demand for the dedupe set, which is [Estimated] ~\$1,875/month of pure duplicate suppression — Redis replaced it.

FAILURE HANDLING — WHAT BREAKS THIS

- **The 10-events-per-POST batching assumption.** If the SDK posts one event per request, collector and ALB sizing are ~10x off. Cheapest possible check, and it is the first thing I would measure.
- **Tenant skew.** 500 tenants are not uniform. One tenant at 40% of volume produces a hot shard the partition key cannot fix. Detection: per-shard iterator age. Mitigation: dedicated shard set for the top N tenants — which quietly raises the cost model.
- **Reconciliation compares two wrong systems.** If the old pipeline is losing 3%, matching it to within 0.1% means matching its errors. The accept ledger is the third, independent source that breaks the tie — without it the whole migration gate is circular.
- **ClickHouse `ReplacingMergeTree` merges are asynchronous.** A duplicate is visible until the part merges. Dashboards must query `FINAL` or accept transient double-counts; at 50M/day that is a real operational cost I have not benchmarked.
- **The bot classifier is a business decision wearing an engineering costume.** Tightening it lowers every customer's reported traffic. It must be versioned, and changes must be announced.
- **The fixture is 25 synthetic lines.** It tells me which *classes* of defect exist. It tells me nothing about their real frequency, and I have not treated it as if it did.

WHAT STAYS HUMAN

- **Executing a GDPR erasure.** Automate discovery, cascade computation, and verification; require a human to authorize the irreversible step and sign the completion record. An automated deleter with a subject-resolution bug destroys the wrong customer's data with no undo.
- **Per-tenant cutover go/no-go.** Rollback is automatic (fast, safe, reversible). Rolling *forward* again is human — an auto-retry loop that flaps 500 tenants' dashboards is worse than the outage.
- **Bot and PII classifier rule changes.** Both silently change numbers customers see and data we are legally accountable for. Code review plus a named owner, never config drive-by.
- **What "accurate" means.** The 0.1% / 14-day gate is a judgment call about acceptable customer risk, not a metric. A human sets it, and a human decides when a tenant is an exception.

ARTIFACT ACCESS

Everything is in the attached `beat-claude-engineer-004/` folder — no login, no network, no dependencies beyond Python 3.9+.

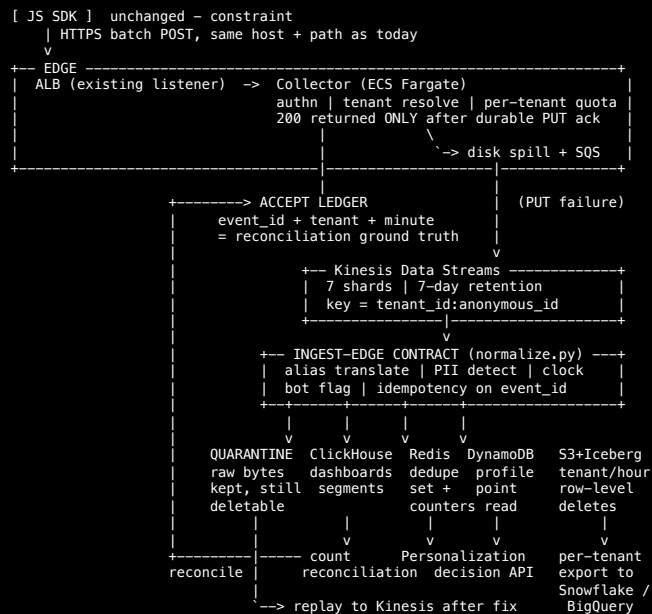
```
$ cd beat-claude-engineer-004
$ shasum -a 256 fixtures/event_sample.jsonl # must match the checksum above
$ python3 detect_anomalies.py               # per-line verdicts + cross-event findings
$ python3 -m unittest discover -s tests -v   # 20 tests
$ python3 bench_ingest.py                   # throughput, on your hardware
$ python3 cost_model.py                     # line-item cost model
```

`out/` holds the committed output of each command as run on my machine (Darwin arm64, Python 3.9.6) so you can diff your run against mine. Diagrams are in `ARCHITECTURE.md` (Mermaid, renders on GitHub). I am happy to walk through any of it live, or re-run it against a changed constraint.

Appendix – Architecture Diagrams

Diagrams do not count toward the 4-page limit. Mermaid source for all four is in `ARCHITECTURE.md` in the artifact folder and renders on GitHub.

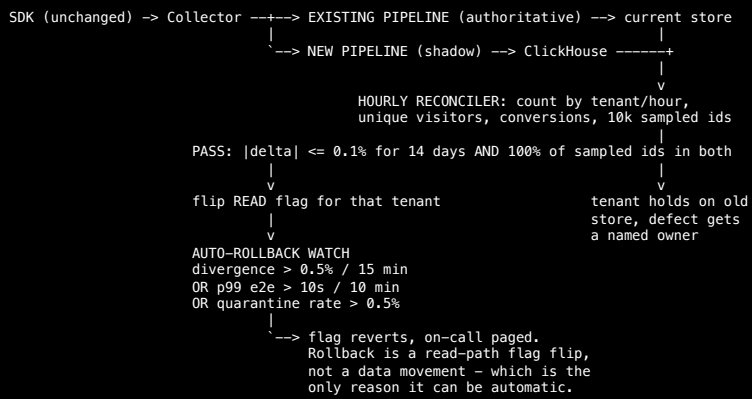
A. DATA FLOW - SDK TO DASHBOARD (nothing on this path may change for the customer)



B. VERDICT MACHINE - one line in, exactly one verdict out (no fifth outcome)

[illegible]

C. MIGRATION – parallel run, per-tenant gate, automatic rollback



D. GDPR DELETION BLAST RADIUS [evt-0017: delete_all_data, GDPR, t-088, u-1077]

1. resolve subject keys: u-1077 + EVERY anonymous_id ever stitched to it
-> catches evt-0006, which carries user_id: null and is reachable only through anon-77a. Deleting by user_id alone leaves it behind.
2. destroy per-subject data key (crypto-shred: PII unreadable in every copy, including backups, without rewriting them)
3. Iceberg row-level delete by subject key
4. delete DynamoDB profile + Redis counters
5. SWEEP THE QUARANTINE TABLE – the second place deletion jobs forget.
evt-0011-class rows have no tenant_id at all.
6. emit tombstone to Kinesis so warehouse exports replay the delete
7. HUMAN sign-off before the destructive step; signed completion record